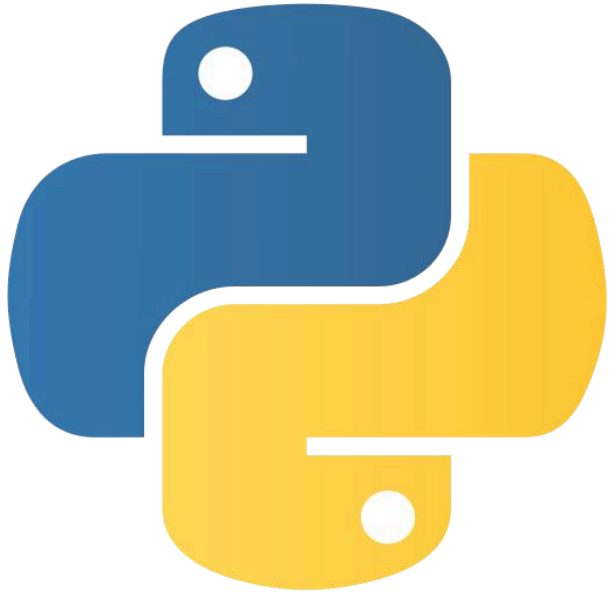
1

CLASE LIVE

Gestión de dependencias con “uv” y Programación Orientada a Objetos básica

Módulo de Programación Avanzada



Álvaro Solano Jiménez

2026 - version 1.0

pontIA

Contenido

01 Gestión de dependencias con “uv”

02 Ejercicios de “uv”

03 POO: repaso de conceptos clave

04 Ejercicio de POO

01 Gestión de dependencias con “uv”



https://docs.astral.sh/uv/

¿Qué es uv?

Gestor de paquetes y proyectos para Python, desarrollado por Astral (creadores de Ruff) y escrito en Rust. Unifica en una sola herramienta lo que antes requería pip, virtualenv, pip-tools, pipx, poetry y pyenv: instala y gestiona dependencias, crea entornos virtuales y gestiona versiones de Python.

01 Gestión de dependencias con “uv”

VENTAJAS

- Extremadamente rápido: 10–100× más veloz que pip gracias a Rust y a una caché global de paquetes
- Todo en uno: reemplaza pip, virtualenv, poetry, pipx y pyenv con una única CLI
- Reproducibilidad: lockfile universal (uv.lock) que garantiza los mismos entornos en cualquier máquina
- Gestiona las propias versiones de Python (*uv python install 3.12*), sin necesidad de instalarlas aparte
- Ejecución de scripts y herramientas sin instalación previa (uv run)

DESVENTAJAS

- Herramienta joven (2024): evoluciona muy rápido y algunas funciones aún cambian entre versiones
- Depende de una empresa privada (Astral), aunque es open source (licencia MIT/Apache)
- Menos documentación, tutoriales y respuestas en la comunidad que pip o poetry
- Algunos flujos muy específicos de pip/conda (p. ej. entornos conda, plugins) no tienen equivalente directo

01 Gestión de dependencias con “uv”

Funcionamiento de la caché al instalar paquetes con diferentes gestores:

	PIP	PIPENV	POETRY	CONDA / MAMBA	UV
Qué cachea	Solo descargas (wheels comprimidos)	Lo mismo que “pip” (lo usa por debajo)	Descargas + metadatos de PyPI	Paquetes ya descomprimidos	**Paquetes ya descomprimidos + metadatos**
Instalación en cada venv	Descomprime y copia todo	Descomprime y copia todo	Descomprime y copia todo	Hard links a la caché global	**Hard links a la caché global**
Espacio con N proyectos	N copias completas	N copias completas	N copias completas	1 copia física compartida	**1 copia física compartida**
Resolutor	Python	Python	Python	C++	**Rust**
Velocidad de reinstalación	Lenta	Lenta	Media (desde que usan su propio instalador)	Rápida	**Muy rápida**
Gestiona versiones de Python	No	No	No	Sí	**Sí**

02 Ejercicios de “uv”

Vamos al código para realizar dos ejercicios:

- Ejercicio 1: crear un proyecto desde cero y resolver conflictos entre dependencias
- Ejercicio 2: migrar un proyecto a una versión nueva de Python



03 POO: repaso de conceptos clave

¡Vamos al código!

Abrid el notebook: `“poo_repaso_conceptos_clave.ipynb”`



04 Ejercicio de POO

¡Vamos al código!

Abrid el notebook: `“poo_ejercicio_1.ipynb”`



pontIA

Muchas gracias por vuestra atención

Álvaro Solano Jiménez

www.pontia.tech

alvaro.solano@centricsoftware.com